```
 acknowledged: true,
  insertedIds: {
    '0': 1,
    '1': 2,
    '2': 3,
    '3': 4,
    '4': 5,
    '5': 6,
    '6': 7,
    '7': 8,
    '8': 9,
    '9': 10
  }
}
db.sales.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
db.sales.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
db.sales.find({
    item: "Mochas"
})
{
  _id: 4,
  item: 'Mochas',
  price: 25,
  size: 'Tall',
  quantity: 11,
  date: 2022-02-17T08:00:00.000Z
}
db.sales.find({
    item: "Mochas"
}).explain("executionStats")
{
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 4,
    totalKeysExamined: 0,
    totalDocsExamined: 10,
    executionStages: {
      isCached: false,
```

```
      stage: 'COLLSCAN',
      filter: {
        item: {
          '$eq': 'Mochas'
        }
      },
      nReturned: 1,
      executionTimeMillisEstimate: 0,
      works: 11,
      advanced: 1,
      needTime: 9,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      nss: 'vit.sales',
      direction: 'forward',
      docsExamined: 10
    }
  }, use vit
switched to db vit
db.sales.insertMany([

      { "_id" : 1, "item" : "Americanos", "price" : 5, "size": "Short", "quantity" :
22, "date" : ISODate("2022-01-15T08:00:00Z") },

      { "_id" : 2, "item" : "Cappuccino", "price" : 6, "size": "Short","quantity" :
12, "date" : ISODate("2022-01-16T09:00:00Z") },

      { "_id" : 3, "item" : "Lattes", "price" : 15, "size": "Grande","quantity" : 25,
"date" : ISODate("2022-01-16T09:05:00Z") },

      { "_id" : 4, "item" : "Mochas", "price" : 25,"size": "Tall", "quantity" : 11,
"date" : ISODate("2022-02-17T08:00:00Z") },

      { "_id" : 5, "item" : "Americanos", "price" : 10, "size": "Grande","quantity" :
12, "date" : ISODate("2022-02-18T21:06:00Z") },

      { "_id" : 6, "item" : "Cappuccino", "price" : 7, "size": "Tall","quantity" : 20,
"date" : ISODate("2022-02-20T10:07:00Z") },

      { "_id" : 7, "item" : "Lattes", "price" : 25,"size": "Tall", "quantity" : 30,
"date" : ISODate("2022-02-21T10:08:00Z") },

      { "_id" : 8, "item" : "Americanos", "price" : 10, "size": "Grande","quantity" :
21, "date" : ISODate("2022-02-22T14:09:00Z") },
```

```
      { "_id" : 9, "item" : "Cappuccino", "price" : 10, "size": "Grande","quantity" : 17, "date" : ISODate("2022-02-23T14:09:00Z") },

      { "_id" : 10, "item" : "Americanos", "price" : 8, "size": "Tall","quantity" : 15, "date" : ISODate("2022-02-25T14:09:00Z")}

]);
{
d
  queryShapeHash: '9383BE82251C8C530A346BE0E5F87D3AC8A4657C03B4033215799FE0667948A0',
  command: {
    find: 'sales',
    filter: {
      item: 'Mochas'
    },
    '$db': 'vit'
  },
  serverInfo: {
    host: 'LAPTOP-TMGN63KB',
    port: 27017,
    version: '8.3.2',
    gitVersion: '89f54eb32e217d2f7bfcad50e5919f3033bb9611'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
db.sales.createIndex({
    item: 1
})
item_1
db.sales.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { item: 1 }, name: 'item_1' }
```

```
]
db.sales.find({
    item: "Americanos"
})
{
  _id: 1,
  item: 'Americanos',
  price: 5,
  size: 'Short',
  quantity: 22,
  date: 2022-01-15T08:00:00.000Z
}
{
  _id: 5,
  item: 'Americanos',
  price: 10,
  size: 'Grande',
  quantity: 12,
  date: 2022-02-18T21:06:00.000Z
}
{
  _id: 8,
  item: 'Americanos',
  price: 10,
  size: 'Grande',
  quantity: 21,
  date: 2022-02-22T14:09:00.000Z
}
{
  _id: 10,
  item: 'Americanos',
  price: 8,
  size: 'Tall',
  quantity: 15,
  date: 2022-02-25T14:09:00.000Z
}
db.sales.find({
    item: "Americanos"
}).explain("executionStats")
{
      docsExamined: 4,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 4,
        executionTimeMillisEstimate: 0,
        works: 5,
```

```
        advanced: 4,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        nss: 'vit.sales',
        keyPattern: {
          item: 1
        },
        indexName: 'item_1',
        isMultiKey: false,
        multiKeyPaths: {
          item: []
        },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          item: [
            '["Americanos", "Americanos"]'
          ]
        },
        keysExamined: 4,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0,
        peakTrackedMemBytes: 0
      }
    }
  },
  queryShapeHash:
'9383BE82251C8C530A346BE0E5F87D3AC8A4657C03B4033215799FE0667
948A0',
  command: {
    find: 'sales',
    filter: {
      item: 'Americanos'
    },
    '$db': 'vit'
  },

  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
```

```
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}db.sales.createIndex({
    date: -1
})
date_-1
db.sales.find()
.sort({ date: -1 })
{
  _id: 10,
  item: 'Americanos',
  price: 8,
  size: 'Tall',
  quantity: 15,
  date: 2022-02-25T14:09:00.000Z
}
{
  _id: 9,
  item: 'Cappuccino',
  price: 10,
  size: 'Grande',
  quantity: 17,
  date: 2022-02-23T14:09:00.000Z
}
{
  _id: 8,
  item: 'Americanos',
  price: 10,
  size: 'Grande',
  quantity: 21,
  date: 2022-02-22T14:09:00.000Z
}
{
  _id: 7,
  item: 'Lattes',
  price: 25,
  size: 'Tall',
  quantity: 30,
  date: 2022-02-21T10:08:00.000Z
```

```
}
{
  _id: 6,
  item: 'Cappuccino',
  price: 7,
  size: 'Tall',
  quantity: 20,
  date: 2022-02-20T10:07:00.000Z
}
{
  _id: 5,
  item: 'Americanos',
  price: 10,
  size: 'Grande',
  quantity: 12,
  date: 2022-02-18T21:06:00.000Z
}
{
  _id: 4,
  item: 'Mochas',
  price: 25,
  size: 'Tall',
  quantity: 11,
  date: 2022-02-17T08:00:00.000Z
}
{
  _id: 3,
  item: 'Lattes',
  price: 15,
  size: 'Grande',
  quantity: 25,
  date: 2022-01-16T09:05:00.000Z
}
{
  _id: 2,
  item: 'Cappuccino',
  price: 6,
  size: 'Short',
  quantity: 12,
  date: 2022-01-16T09:00:00.000Z
}
{
  _id: 1,
  item: 'Americanos',
  price: 5,
  size: 'Short',
  quantity: 22,
```

```
  date: 2022-01-15T08:00:00.000Z
}
db.sales.dropIndex("item_1")
{ nIndexesWas: 5, ok: 1 }
db.sales.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { price: 1 }, name: 'price_1' },
  { v: 2, key: { quantity: 1 }, name: 'quantity_1' },
  { v: 2, key: { date: -1 }, name: 'date_-1' }
]
db.users.insertMany([
   { email:  "john@test.com", name: "john"},
   { email:  "jane@test.com", name: "jane"},
]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a241ebf8b07e9ebbc14f413'),
    '1': ObjectId('6a241ebf8b07e9ebbc14f414')
  }
}
db.users.createIndex({email:1},{unique:true});
email_1
db.users.insertOne(
   { email:  "john@test.com", name: "johny"}
);
MongoServerError: E11000 duplicate key error collection: vit.users index:
email_1 dup key: { email: "john@test.com" }
db.users.drop()
true
db.users.insertMany([
   { email:  "john@test.com", name: "john"},
   { email:  "john@test.com", name: "johny"},
   { email:  "jane@test.com", name: "jane"},
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a241eea8b07e9ebbc14f416'),
    '1': ObjectId('6a241eea8b07e9ebbc14f417'),
    '2': ObjectId('6a241eea8b07e9ebbc14f418')
  }
}
db.users.createIndex({email: 1},{unique:true})
MongoServerError[DuplicateKey]: Index build failed: 8ec9ee84-9f70-41bc-
a3dc-1f625c2086e3: Collection vit.users ( e8d9775f-f03c-445d-
```

bdfa-0d338aa19391) :: caused by :: E11000 duplicate key error collection: vit.users index: email_1 dup key: { email: "john@test.com" }
db.users.deleteOne({name:'johny', email: 'john@test.com'});
{
  acknowledged: true,
  deletedCount: 1
}
db.users.createIndex({email: 1},{unique:true})
email_1
db.collection.createIndex({field1: 1, field2: 1}, {unique: true});
field1_1_field2_1
db.locations.insertOne({
        address: "Downtown San Jose, CA, USA",
        lat: 37.335480,
        long: -121.893028
})
{
  acknowledged: true,
  insertedId: ObjectId('6a241f128b07e9ebbc14f419')
}
db.locations.createIndex({
        lat: 1,
        long: 1
},{ unique: true });
lat_1_long_1
db.locations.insertOne({
        address: "Dev Bootcamp, San Jose, CA, USA",
        lat: 37.335480,
        long: -122.893028
})
{
  acknowledged: true,
  insertedId: ObjectId('6a241f238b07e9ebbc14f41a')
}
db.locations.insertOne({
        address: "Central San Jose, CA, USA",
        lat: 37.335480,
        long: -121.893028
})
MongoServerError: E11000 duplicate key error collection: vit.locations index: lat_1_long_1 dup key: { lat: 37.33548, long: -121.893028 }
db.sales.drop()
true
db.sales.insertMany([{ "_id" : 1, "item" : "Americanos", "price" : 5, "size": "Short", "quantity" : 22, "date" : ISODate("2022-01-15T08:00:00Z") }, { "_id" : 2, "item" : "Cappuccino", "price" : 6, "size": "Short","quantity" : 12, "date" : ISODate("2022-01-16T09:00:00Z") }, { "_id" : 3, "item" : "Lattes", "price" : 15,

"size": "Grande","quantity" : 25, "date" : ISODate("2022-01-16T09:05:00Z") }, { "_id" : 4, "item" : "Mochas", "price" : 25,"size": "Tall", "quantity" : 11, "date" : ISODate("2022-02-17T08:00:00Z") }, { "_id" : 5, "item" : "Americanos", "price" : 10, "size": "Grande","quantity" : 12, "date" : ISODate("2022-02-18T21:06:00Z") }, { "_id" : 6, "item" : "Cappuccino", "price" : 7, "size": "Tall","quantity" : 20, "date" : ISODate("2022-02-20T10:07:00Z") }, { "_id" : 7, "item" : "Lattes", "price" : 25,"size": "Tall", "quantity" : 30, "date" : ISODate("2022-02-21T10:08:00Z") }, { "_id" : 8, "item" : "Americanos", "price" : 10, "size": "Grande","quantity" : 21, "date" : ISODate("2022-02-22T14:09:00Z") }, { "_id" : 9, "item" : "Cappuccino", "price" : 10, "size": "Grande","quantity" : 17, "date" : ISODate("2022-02-23T14:09:00Z") }, { "_id" : 10, "item" : "Americanos", "price" : 8, "size": "Tall","quantity" : 15, "date" : ISODate("2022-02-25T14:09:00Z")}
SyntaxError: Unexpected token, expected "," (1:1275)

> 1 | db.sales.insertMany([{ "_id" : 1, "item" : "Americanos", "price" : 5, "size": "Short", "quantity" : 22, "date" : ISODate("2022-01-15T08:00:00Z") }, { "_id" : 2, "item" : "Cappuccino", "price" : 6, "size": "Short","quantity" : 12, "date" : ISODate("2022-01-16T09:00:00Z") }, { "_id" : 3, "item" : "Lattes", "price" : 15, "size": "Grande","quantity" : 25, "date" : ISODate("2022-01-16T09:05:00Z") }, { "_id" : 4, "item" : "Mochas", "price" : 25,"size": "Tall", "quantity" : 11, "date" : ISODate("2022-02-17T08:00:00Z") }, { "_id" : 5, "item" : "Americanos", "price" : 10, "size": "Grande","quantity" : 12, "date" : ISODate("2022-02-18T21:06:00Z") }, { "_id" : 6, "item" : "Cappuccino", "price" : 7, "size": "Tall","quantity" : 20, "date" : ISODate("2022-02-20T10:07:00Z") }, { "_id" : 7, "item" : "Lattes", "price" : 25,"size": "Tall", "quantity" : 30, "date" : ISODate("2022-02-21T10:08:00Z") }, { "_id" : 8, "item" : "Americanos", "price" : 10, "size": "Grande","quantity" : 21, "date" : ISODate("2022-02-22T14:09:00Z") }, { "_id" : 9, "item" : "Cappuccino", "price" : 10, "size": "Grande","quantity" : 17, "date" : ISODate("2022-02-23T14:09:00Z") }, { "_id" : 10, "item" : "Americanos", "price" : 8, "size": "Tall","quantity" : 15, "date" : ISODate("2022-02-25T14:09:00Z")}
    |
^
db.sales.insertMany([
      { "_id" : 1, "item" : "Americanos", "price" : 5, "size": "Short", "quantity" : 22, "date" : ISODate("2022-01-15T08:00:00Z") },
      { "_id" : 2, "item" : "Cappuccino", "price" : 6, "size": "Short","quantity" : 12, "date" : ISODate("2022-01-16T09:00:00Z") },
      { "_id" : 3, "item" : "Lattes", "price" : 15, "size": "Grande","quantity" : 25, "date" : ISODate("2022-01-16T09:05:00Z") },
      { "_id" : 4, "item" : "Mochas", "price" : 25,"size": "Tall", "quantity" : 11, "date" : ISODate("2022-02-17T08:00:00Z") },
      { "_id" : 5, "item" : "Americanos", "price" : 10, "size": "Grande","quantity" : 12, "date" : ISODate("2022-02-18T21:06:00Z") },

```
       { "_id" : 6, "item" : "Cappuccino", "price" : 7, "size": "Tall","quantity" : 20,
"date" : ISODate("2022-02-20T10:07:00Z") },
       { "_id" : 7, "item" : "Lattes", "price" : 25,"size": "Tall", "quantity" : 30,
"date" : ISODate("2022-02-21T10:08:00Z") },
       { "_id" : 8, "item" : "Americanos", "price" : 10, "size": "Grande","quantity" :
21, "date" : ISODate("2022-02-22T14:09:00Z") },
       { "_id" : 9, "item" : "Cappuccino", "price" : 10, "size": "Grande","quantity" :
17, "date" : ISODate("2022-02-23T14:09:00Z") },
       { "_id" : 10, "item" : "Americanos", "price" : 8, "size": "Tall","quantity" : 15,
"date" : ISODate("2022-02-25T14:09:00Z")}
]);
{
  acknowledged: true,
  insertedIds: {
    '0': 1,
    '1': 2,
    '2': 3,
    '3': 4,
    '4': 5,
    '5': 6,
    '6': 7,
    '7': 8,
    '8': 9,
    '9': 10
  }
}
db.movies.createIndex({ title: 1, year: 1 })

db.movies.find({
    title: /valley/gi,
    year: 2014
}).explain("executionStats")

db.movies.find({
    title: /valley/gi
}).explain("executionStats")

db.movies.find({
    year: 2014
}).explain("executionStats")
{
        }
      },
      nss: 'vit.movies',
      direction: 'forward'
    },
    rejectedPlans: []
```

```
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 0,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 0,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        year: {
          '$eq': 2014
        }
      },
      nReturned: 0,
      executionTimeMillisEstimate: 0,
      works: 1,
      advanced: 0,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      nss: 'vit.movies',
      direction: 'forward',
      docsExamined: 0
    }
  },
  queryShapeHash:
'B11C8E34838EF05C70CBB7A39AC455D6881D6FF06D32B608BA96EE611794
FE56',
  command: {
    find: 'movies',
    filter: {
      year: 2014
    },
    '$db': 'vit'
  },

  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
```

```
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
db.sales.createIndex({
    item: 1,
    price: 1
})
item_1_price_1
db.sales.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { item: 1, price: 1 }, name: 'item_1_price_1' }
]
db.movies.find({
    title: /valley/gi,
    year: 2014
})
db.sales.find({
    item: /americano/i,
    price: 10
}).explain("executionStats")
{
        indexName: 'item_1_price_1',
        isMultiKey: false,
        multiKeyPaths: {
          item: [],
          price: []
        },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          item: [
            '["", {})',
            '[/americano/i, /americano/i]'
          ],
          price: [
            '[10, 10]'
          ]
        },
        keysExamined: 7,
```

```
        seeks: 5,
        dupsTested: 0,
        dupsDropped: 0,
        peakTrackedMemBytes: 0
      }
    }
  },
  queryShapeHash:
'53940AB8E4259CE3BB68093E121147B2A71973BF3946178A4989333BCECB
133A',
  command: {
    find: 'sales',
    filter: {
      item: BSONRegExp('americano', 'i'),
      price: 10
    },
    '$db': 'vit'
  },

  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
winningPlan: {
    stage: "FETCH",
    inputStage: {
        stage: "IXSCAN",
        indexName: "item_1_price_1"
    }
}
SyntaxError: Missing semicolon. (3:14)

  1 | winningPlan: {
  2 |     stage: "FETCH",
> 3 |     inputStage: {
    |                ^
  4 |         stage: "IXSCAN",
```

```
  5 |         indexName: "item_1_price_1"
  6 |     }
db.movies.find({
    title: /valley/gi,
    year: 2014
})
db.movies.find({
    title: /valley/gi
}).explain("executionStats")
BSONError: The regular expression option [g] is not supported
db.sales.find({
    item: /americano/i
}).explain("executionStats")
{
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        nss: 'vit.sales',
        keyPattern: {
          item: 1,
          price: 1
        },
        indexName: 'item_1_price_1',
        isMultiKey: false,
        multiKeyPaths: {
          item: [],
          price: []
        },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          item: [
            '["", {})',
            '[/americano/i, /americano/i]'
          ],
          price: [
            '[MinKey, MaxKey]'
          ]
        },
        keysExamined: 10,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0,
        peakTrackedMemBytes: 0
```

```
      }
    }
  },
  queryShapeHash:
'986BC2DE3FB9448ADF91B540AA1BAAFAFDFF9343A209C5A01468BDF7F6F
F9B4A',
  command: {
    find: 'sales',
    filter: {
      item: BSONRegExp('americano', 'i')
    },
    '$db': 'vit'
  },
,
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
db.sales.find({
    price: 10
}).explain("executionStats")
{
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 10,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        price: {
          '$eq': 10
        }
      },
      nReturned: 3,
      executionTimeMillisEstimate: 0,
      works: 11,
      advanced: 3,
```

```
      needTime: 7,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      nss: 'vit.sales',
      direction: 'forward',
      docsExamined: 10
    }
  },
  queryShapeHash:
'801B3B7F35DE5F86C802A41583BFF70A4D31BD66DF2324D0016137751EFC
0E68',
  command: {
    find: 'sales',
    filter: {
      price: 10
    },
    '$db': 'vit'
  },

  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
db.sales.createIndex({
    size: 1,
    quantity: 1
})
size_1_quantity_1
db.sales.find({
    size: "Grande",
    quantity: { $gt: 15 }
}).explain("executionStats")
{
        isPartial: false,
        indexVersion: 2,
```

```
        direction: 'forward',
        indexBounds: {
          size: [
            '["Grande", "Grande"]'
          ],
          quantity: [
            '(15, inf]'
          ]
        },
        keysExamined: 3,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0,
        peakTrackedMemBytes: 0
      }
    }
  },
  queryShapeHash:
'9150B00F62AED3BDEA9CF2E03D26A2ED1FB8C114B70682DEEB383A10463
A6DDE',
  command: {
    find: 'sales',
    filter: {
      size: 'Grande',
      quantity: {
        '$gt': 15
      }
    },
    '$db': 'vit'
  },

  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
db.sales.createIndex({
    item: 1,
```

```
    size: 1
})
item_1_size_1
db.sales.find({
    item: "Americanos",
    size: "Grande"
}).explain("executionStats")
{
        indexName: 'item_1_size_1',
        isMultiKey: false,
        multiKeyPaths: {
          item: [],
          size: []
        },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          item: [
            '["Americanos", "Americanos"]'
          ],
          size: [
            '["Grande", "Grande"]'
          ]
        },
        keysExamined: 2,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0,
        peakTrackedMemBytes: 0
      }
    }
  },
  queryShapeHash:
'303E34BF2FE739FF4FFA49F94F00D16DABCBD4D0FBA695F323B1C759143
D7B68',
  command: {
    find: 'sales',
    filter: {
      item: 'Americanos',
      size: 'Grande'
    },
    '$db': 'vit'
  },
```

```
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
db.sales.find({
    price: 10
}).explain("executionStats")
{
      filter: {
        price: {
          '$eq': 10
        }
      },
      nss: 'vit.sales',
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 3,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 10,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        price: {
          '$eq': 10
        }
      },
      nReturned: 3,
      executionTimeMillisEstimate: 0,
      works: 11,
      advanced: 3,
      needTime: 7,
```

```
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      nss: 'vit.sales',
      direction: 'forward',
      docsExamined: 10
    }
  },
  queryShapeHash:
'801B3B7F35DE5F86C802A41583BFF70A4D31BD66DF2324D0016137751EFC
0E68',
  command: {
    find: 'sales',
    filter: {
      price: 10
    },
    '$db': 'vit'
  },

  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
db.sales.dropIndex("price_1")
{ nIndexesWas: 4, ok: 1 }
db.sales.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { item: 1, price: 1 }, name: 'item_1_price_1' },
  { v: 2, key: { size: 1, quantity: 1 }, name: 'size_1_quantity_1' },
  { v: 2, key: { item: 1, size: 1 }, name: 'item_1_size_1' }
]
db.sales.dropIndex("quantity_-1")
{ nIndexesWas: 4, ok: 1 }
db.sales.dropIndex({
    quantity: -1
```

```
})
{ nIndexesWas: 4, ok: 1 }
db.sales.createIndex({ item: 1 })

db.sales.createIndex({ price: 1 })

db.sales.createIndex({ quantity: 1 })
quantity_1
db.sales.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { item: 1, price: 1 }, name: 'item_1_price_1' },
  { v: 2, key: { size: 1, quantity: 1 }, name: 'size_1_quantity_1' },
  { v: 2, key: { item: 1, size: 1 }, name: 'item_1_size_1' },
  { v: 2, key: { item: 1 }, name: 'item_1' },
  { v: 2, key: { price: 1 }, name: 'price_1' },
  { v: 2, key: { quantity: 1 }, name: 'quantity_1' }
]
db.sales.dropIndexes()
{
  nIndexesWas: 7,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
db.sales.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
db.sales.drop()
```